**TCS NQT Coding Problem – Full Solution Format**

**1. Problem Title & Source**

Title: **Golden House Coin Puzzle**

Source: **TCS NQT Previous Coding Questions**

**2. Problem Statement (Clear Version)**

In the kingdom of Eldoria, a row of **N golden houses** contains coins.

Each house contains a certain number of coins.

The king announces a puzzle:

A player can choose **any continuous group of houses** and collect all coins from those houses.

However, the player wants to choose the group of houses such that the **total coins collected are maximized**.

Your task is to determine the **maximum number of coins that can be collected from any continuous group of houses**.

In simple terms:

- You are given an array representing coins in houses.
- You must find the **maximum possible sum of a contiguous subarray**.

**3. Examples**

**Example 1**

Input

5

1 2 3 -2 5

Output

9

Explanation

Possible continuous groups:

1 → 1

1 2 → 3

1 2 3 → 6

1 2 3 -2 → 4

1 2 3 -2 5 → 9

2 3 -2 5 → 8

3 -2 5 → 6

5 → 5

Maximum coins collected:

9

**Example 2**

Input

4

-1 -2 -3 -4

Output

-1

Explanation

All numbers are negative.

The best choice is:

[-1]

Maximum sum = **-1**

**4. Constraints**

$1 \le N \le 100$

$-1000 \le arr[i] \le 1000$

Observations

- Values can be **negative**
- Continuous selection is required

**5. Core Concept (Pattern)**

**Maximum Subarray Problem**

Algorithm used:

Kadane's Algorithm

This is a **classic dynamic programming optimization problem**.

**6. Thought Process**

**Brute Force Idea**

Check all possible subarrays.

For each starting index:

calculate sum of every possible subarray

Time complexity

$O(N^2)$

**Optimized Idea (Kadane's Algorithm)**

Key idea:

If the running sum becomes **negative**, it cannot contribute to a maximum sum later.

So we **reset the running sum**.

Steps:

1. Maintain current sum
2. Update maximum sum
3. Reset current sum if it becomes negative

**7. Visualization**

Example

[1, 2, 3, -2, 5]

Process

| Element | Current Sum | Max Sum |
|---|---|---|
| 1 | 1 | 1 |
| 2 | 3 | 3 |
| 3 | 6 | 6 |
| -2 | 4 | 6 |
| 5 | 9 | 9 |

Final result

9

**8. Pseudocode**

```
read n
read array

currentSum = arr[0]
maxSum = arr[0]

for i from 1 to n-1

   currentSum = max(arr[i], currentSum + arr[i])

   maxSum = max(maxSum, currentSum)

print maxSum
```

**9. Code Implementation**

**Python**

```
n = int(input())
```

```
arr = list(map(int,input().split()))

current_sum = arr[0]
max_sum = arr[0]

for i in range(1,n):

    current_sum = max(arr[i], current_sum + arr[i])

    max_sum = max(max_sum, current_sum)

print(max_sum)
```

**Java**

```
import java.util.*;

class Main {

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        int[] arr = new int[n];

        for(int i=0;i<n;i++)
            arr[i] = sc.nextInt();

        int currentSum = arr[0];
        int maxSum = arr[0];

        for(int i=1;i<n;i++){

            currentSum = Math.max(arr[i], currentSum + arr[i]);

            maxSum = Math.max(maxSum, currentSum);
```

```
        }

        System.out.println(maxSum);
    }
}
```

**10. Time & Space Complexity**

**Time Complexity**

O(N)

Single pass through the array.

**Space Complexity**

O(1)

Only constant variables used.

**11. Common Mistakes**

1. Initializing sum with **0 instead of arr[0]**
2. Ignoring the **all-negative array case**
3. Using nested loops unnecessarily
4. Forgetting to update max sum

**12. Detailed Dry Run**

Input

1 2 3 -2 5

| Index | Value | Current Sum | Max Sum |
|---|---|---|---|
| 0 | 1 | 1 | 1 |
| 1 | 2 | 3 | 3 |
| 2 | 3 | 6 | 6 |
| 3 | -2 | 4 | 6 |
| 4 | 5 | 9 | 9 |

Output

9

**13. Real-Life Applications**

Maximum subarray problems appear in:

- Stock market profit analysis

- Signal processing
- Profit/loss analysis
- Maximum revenue segment detection

**14. Common Traps**

1. Treating it as **prefix sum**
2. Resetting incorrectly
3. Missing negative cases
4. Incorrect initialization

**15. Related Problems**

Classic problems using Kadane's algorithm:

LC 53 Maximum Subarray

LC 918 Maximum Circular Subarray

LC 152 Maximum Product Subarray

**16. Alternate Approaches**

| Approach | Complexity | Notes |
|---|---|---|
| Brute Force | $O(N^2)$ | check all subarrays |
| Kadane's Algorithm | O(N) | optimal |

**17. Why This Solution Works**

If a prefix sum becomes negative, including it will only **reduce the total sum of future subarrays**.

Thus we discard negative prefixes and continue with the next element.

This greedy decision ensures we always maintain the **best possible running sum**.

**18. Variations / Follow-Up Questions**

Possible interview variations:

1. Maximum **circular subarray sum**
2. Maximum **product subarray**
3. Maximum subarray with **length ≤ K**
4. Maximum **non-overlapping subarrays**